Course Project: Graph Analytics Orchestrator

Oct 13, 2026 · @John

Overview

In groups of about 5, you build an LLM-based orchestrator that answers natural-language questions about graphs by planning, calling and chaining tools from the class registry built in Assignment 3. The project runs from **Friday, October 30** to **Friday, November 6, 11:59 PM IST**.

You may use any tool in the registry. Each registry tool follows the tool contract in the Assignment 3 handout.

Key dates

Date	What
Fri, Oct 30	Groups announced; registry will be released
Fri, Nov 6, 11:59 PM IST	Final submission due

What you build

Your orchestrator must:

1. Accept a natural-language question about a provided graph.
2. Discover tools by reading registry manifests at startup, not by hard-coding them. Choose which implementations to load by performing tests. Keep only those tools that you choose in your registry.
3. Take every fact about the graph from a tool result. An answer the trace does not support counts as wrong.
4. Answer multi-step questions by chaining tools through `result_graph_id`.
5. Recognise questions that are ambiguous or that no available tool supports, and say so.
6. Log a trace for every question: each LLM call, each tool call with its arguments and output, and the final answer.
7. Run with one command, `python run_queries.py --queries <file> --out answers.json`, reading the API key from an environment variable.
8. Try to keep the tool calls per question to a minimum. Usually, no more than five calls will be required.

Build the orchestrator either in plain Python (your own agent loop) or with LangGraph/LangChain. No other agent frameworks.

Groups and roles

Each member leads one area, and everyone writes code. The group with just four members can merge Tool curator into Evaluation lead. (The roles are a recommendation, not a requirement. Do what works for your group.)

Role	Leads on
Planner	Prompts and logic that turn a question into a plan of tool calls
Executor	Running tool calls, passing results between steps, handling errors
Tool curator	Choosing and testing registry implementations
Evaluation lead	Running the query sets, measuring accuracy, analysing failures
Integration and demo lead	The runner script and README

Answers and query sets

For each question, your orchestrator writes one entry to `answers.json`:

```
{
  "query_id": "q07",
  "status": "answered",
  "answer_text": "Alice is the most influential member of the largest
community.",
  "answer": {"type": "node", "value": "n42"},
  "tools_used": ["louvain.s045", "pagerank.s123"],
  "trace_file": "traces/q07.jsonl"
}
```

`status` is `answered`, `clarification_needed`, `unsupported` or `error`. `answer.type` is `number`, `boolean`, `node`, `node_list`, `path` or `text`.

Answers are compared using the structured `answer` field. Numbers must match within 0.1%; node lists are compared as sets, or in order for rankings; ambiguous and unsupported questions need the right `status`. An answer the trace does not support

counts as wrong.

- **Public set** (October 30): about 30 questions with expected answers, mixing single-tool, multi-step, ambiguous and unsupported questions.
- **Evaluation set** (hidden): about 30 new questions of the same kinds. After the deadline, the instructors run your submitted code on it with the one command above, so make sure it runs from a clean setup.

Deliverables

Each group submits one zip file on Moodle by **Friday, November 6, 11:59 PM IST**. There are no intermediate submissions. The zip contains:

1. Your code, with a README that covers setup and, importantly, how to run it. Keep the running mechanism as simple as possible. Explicitly mention the environment variable name (not the API key) in the README file. Use the same directory structure that you used for Assignment 3.
2. `answers_public.json` and the `traces/` folder for the public set.
3. A report of at most 4 pages: your design, results on the public set, an analysis of at least three failures, and a short section saying what each member contributed.

Late submissions will not be accepted.

Rules

- Code is shared within your group only. Do not share any code with other groups. However, you are free to discuss ideas across groups.
- Post questions in the General Discussion forum on Moodle. Feel free to answer questions.
- Use free tools only, including an LLM with a free tier that supports function calling. Suggestion (optional): to stay within rate limits, test on a few questions at a time, and consider caching LLM responses so that repeated runs reuse them.
- Never commit an API key. Read it from an environment variable.
- Do not edit registry tools. Report bugs in the General Discussion forum on Moodle.
- You may use free-tier AI coding tools. Acknowledge any such use in your report. You must be able to explain any code you submit.